

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5 Total Marks:50 Annexure A

Scenario Based Assignment Code of Conduct:

*I SHAILAJA S (192511026) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms

- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

i. Greedy Best-First Search (GBFS)

GBFS selects the path with the smallest heuristic value $h(n)$, such as the shortest straight-line distance to the destination. It ignores the actual travel cost already incurred. In this scenario, the drone quickly chooses the route that appears closest to the victims.

Strengths:

- Very fast decision-making.
- Low memory usage.
- Useful when urgent action is required.

Weaknesses:

- May choose flooded or risky routes because it ignores actual travel cost.
- Not guaranteed to find the optimal path.
- Requires frequent replanning when roads become blocked.

ii. A* Search

A* evaluates nodes using $f(n)=g(n)+h(n)$, where $g(n)$ is the actual cost from the start and $h(n)$ is the estimated cost to the goal. It balances the distance already travelled with the estimated remaining distance.

Advantages:

- Finds the optimal path when the heuristic is admissible.
- Considers terrain, weather, and blocked roads.
- Produces safer and more reliable routes.

Disadvantage:

- Uses more memory and computation than GBFS.

iii. Impact of Heuristic Accuracy

An accurate heuristic improves both algorithms. GBFS becomes faster but still may miss the optimal path. A* becomes both efficient and optimal. If the heuristic is poor, GBFS may repeatedly choose bad routes, while A* still performs better because it also considers actual travel cost.

iv. Effect of Dynamic Changes

When roads suddenly become blocked, both algorithms must replan. GBFS may frequently abandon its chosen path and restart, causing delays. A* updates its search using accumulated costs and generally adapts more effectively, although replanning increases computation time.

v. Recommendation

A* is the most suitable algorithm because it balances speed, optimality, and safety. Although it requires more computation than GBFS, it minimizes overall travel cost, avoids risky routes, adapts better to dynamic changes, and is more reliable for emergency medicine delivery.

Comparison Table

Feature	GBFS	A*
Evaluation	$h(n)$	$g(n)+h(n)$
Optimal	No	Yes
Speed	Very Fast	Moderate
Safety	Lower	Higher
Dynamic changes	Poor	Better

2. **Scenario:** A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations ii.

Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

- iv. Recommend the best approach and justify your choice based on scalability and adaptability

Optimization Problem & Why Traditional Search is Inefficient

The optimization problem is to find the **best traffic signal timings** that reduce waiting time, fuel consumption, and congestion while satisfying traffic rules.

Traditional search algorithms (BFS, DFS) are inefficient because:

- The search space is extremely large.
- Traffic conditions change dynamically.
- They require high computation time and memory.
- They are not suitable for real-time traffic optimization.

i. Hill Climbing Algorithm

Hill Climbing starts with an initial traffic signal configuration and continuously selects a neighboring solution that improves traffic flow.

Advantages

- Simple and fast.
- Low memory usage.
- Suitable for quick local optimization.

Limitations

- Gets stuck at local maxima.
- Cannot guarantee the best solution.
- Performs poorly in dynamic environments.

ii. Local Maxima, Plateaus, and Ridges

Local Maxima

A solution that is better than nearby solutions but not the global best.

Example: One intersection improves, but overall city traffic remains congested.

Plateau

A flat region where neighboring solutions have the same performance.

Example: Small changes in signal timing produce no improvement.

Ridge

A solution where multiple simultaneous changes are needed for improvement.

Example: Coordinated timing changes at several intersections are required to reduce congestion.

iii. Simulated Annealing and Genetic Algorithm

Simulated Annealing

- Occasionally accepts worse solutions to escape local maxima.
- Finds better solutions than Hill Climbing.
- Suitable for dynamic environments.

Genetic Algorithm (GA)

- Works with multiple solutions simultaneously.
- Uses selection, crossover, and mutation to generate better solutions.
- Produces near-optimal results for large search spaces.

iv. Recommended Approach

Genetic Algorithm is the most suitable because:

- Handles large-scale traffic networks efficiently.
- Adapts to changing traffic conditions.
- Escapes local optima.
- Provides high-quality and scalable solutions.
- Minimizes waiting time, fuel consumption, and congestion.

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing

capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments
- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

i. Why Online Search is Required Instead of Offline Search

An **online search agent** is required because the rover does not know the complete environment before starting. It makes decisions based on the information gathered while exploring.

Why Offline Search is Not Suitable:

- No complete map is available.
- Hazards are discovered during exploration.
- The environment is uncertain and changes continuously.
- Real-time decision-making is required.

ii. Challenges of Partial Observability and Dynamic Environment

Partial Observability

- The rover can sense only nearby areas.
- Hidden obstacles may exist beyond its sensing range.
- Decisions must be made with incomplete information.

Dynamic Environment

- Dust storms and terrain conditions may change.
- New obstacles can appear unexpectedly.
- Previously safe paths may become unsafe.

These challenges increase uncertainty and require continuous replanning.

iii. Building and Updating the Internal Model

The rover creates an internal map while exploring.

Steps:

1. Sense nearby environment.
2. Detect obstacles and safe paths.
3. Update the internal map.
4. Plan the next movement.
5. Repeat until the destination is reached.

iv. Comparison of Search Strategies

Feature	Uninformed Search	Informed Search	Online Search
Environment Knowledge	None	Uses Heuristic	Learns During Search
Complete Map Needed	Yes	Yes	No
Handles Unknown Terrain	No	Limited	Yes
Adaptability	Low	Moderate	High
Suitable for Mars Rover	No	Partly	Yes

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii.

Explain how backtracking search works in this scenario

- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

i. Variables, Domains, and Constraints

Variables

Variables represent the exams that need to be scheduled.

Example:

- E1 = Mathematics
- E2 = Physics
- E3 = Chemistry
- E4 = Computer Science

Domains

The domain is the set of possible values for each variable.

Example:

- Time Slots: Morning, Afternoon, Evening
- Exam Halls: Hall A, Hall B, Hall C

Each exam can be assigned to any available time slot and hall.

Constraints

The timetable must satisfy the following constraints:

- No student should have two exams at the same time.
- Limited exam halls should not exceed capacity.
- Some subjects must be conducted before others.
- Faculty should be available during the assigned exam time.
- Students requiring special accommodations must receive suitable exam halls.

These constraints ensure a valid and conflict-free timetable.

ii. Backtracking Search

Backtracking is a search algorithm used to solve CSPs by assigning values one by one.

Working

1. Assign a time slot and hall to an exam.
2. Check whether all constraints are satisfied.
3. If valid, continue assigning the next exam.
4. If a conflict occurs, return (backtrack) and try another assignment.
5. Repeat until all exams are scheduled.

Advantages

- Finds a valid timetable.
- Avoids invalid schedules.
- Guarantees a solution if one exists.

Disadvantage

- Can be slow when the number of exams is very large.

iii. Constraint Propagation (Forward Checking)

Constraint propagation reduces the search space by eliminating invalid choices early.

Forward Checking

After assigning a value to one exam, the algorithm removes conflicting values from the remaining exams.

Example

If **Mathematics** is scheduled on **Monday Morning**, then another exam with the same students cannot be scheduled in that time slot.

This reduces unnecessary searching and speeds up timetable generation.

Advantages

- Detects conflicts early.
- Reduces backtracking.
- Improves search efficiency.
- Generates solutions faster.

iv. Real-World Challenges and Best Strategy

Challenges

- Large number of students and courses.
- Limited exam halls.
- Faculty availability changes.
- Last-minute timetable modifications.
- Special accommodation requirements.

5.Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

i. Minimax Algorithm

The **Minimax algorithm** is used for two-player games where one player tries to maximize the score (AI) while the opponent tries to minimize it.

Working

- AI (MAX) chooses the move with the highest score.
- Opponent (MIN) chooses the move with the lowest score.
- The AI selects the move that gives the best possible outcome assuming the opponent plays optimally.

ii. Computational Challenges

The game has a very large decision tree because:

- Many possible moves at each stage.
- Search depth increases rapidly.
- High computation time and memory usage.
- Difficult to make decisions in real time.

iii. Alpha-Beta Pruning

Alpha-Beta Pruning improves Minimax by removing branches that cannot affect the final decision.

Advantages

- Reduces unnecessary node evaluation.
- Speeds up decision making.
- Produces the same optimal result as Minimax.
- Allows deeper search within limited time.

iv. Evaluation Function

The evaluation function estimates the quality of a game state.

Factors Considered

- Number of remaining units.
- Health of units.
- Resources collected.
- Territory controlled.
- Distance from enemy.
- Defensive and attacking strength.

A higher score indicates a better position for the AI.

v. Recommended Strategy

The best approach is **Minimax with Alpha-Beta Pruning** because:

- Finds optimal moves.
- Reduces computation time.
- Handles large game trees efficiently.
- Suitable for real-time strategy games.
- Balances accuracy and fast decision making.

RUBRICS FOR CALCULATION

Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)
20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects
20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points
25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application
20%	Makes well reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification

Criteria Level 1 (Beginning)

Misinterprets
Understanding
of Scenario

Response 15%

Application of
Concepts

Decision
Making

Identification of
Key Issues

Clarity of

Response is clear, scenario structured, and logically presented

Unable to identify key issues

Unable to apply relevant concepts

No clear decisions made or rationale provided

Disorganized and difficult to understand

Generally clear with minor issues

Somewhat unclear or poorly structured or fails to understand the